

## GUÍA PRÁCTICA DE LABORATORIO

### Frontend en Angular 20 con Bootstrap 5 para consumir una API REST

| Datos generales   |                                                                                        |
|-------------------|----------------------------------------------------------------------------------------|
| Curso             | JavaScript Avanzado                                                                    |
| Tema              | Consumo de una API REST desde Angular: servicios HTTP, formularios reactivos y signals |
| Modalidad         | Laboratorio (práctica guiada en clase)                                                 |
| Duración estimada | 180 minutos                                                                            |
| Docente           | Ing. David Reyna Barreto                                                               |
| Estudiante        |                                                                                        |
| Sección / Fecha   |                                                                                        |

## 1. Logro de aprendizaje

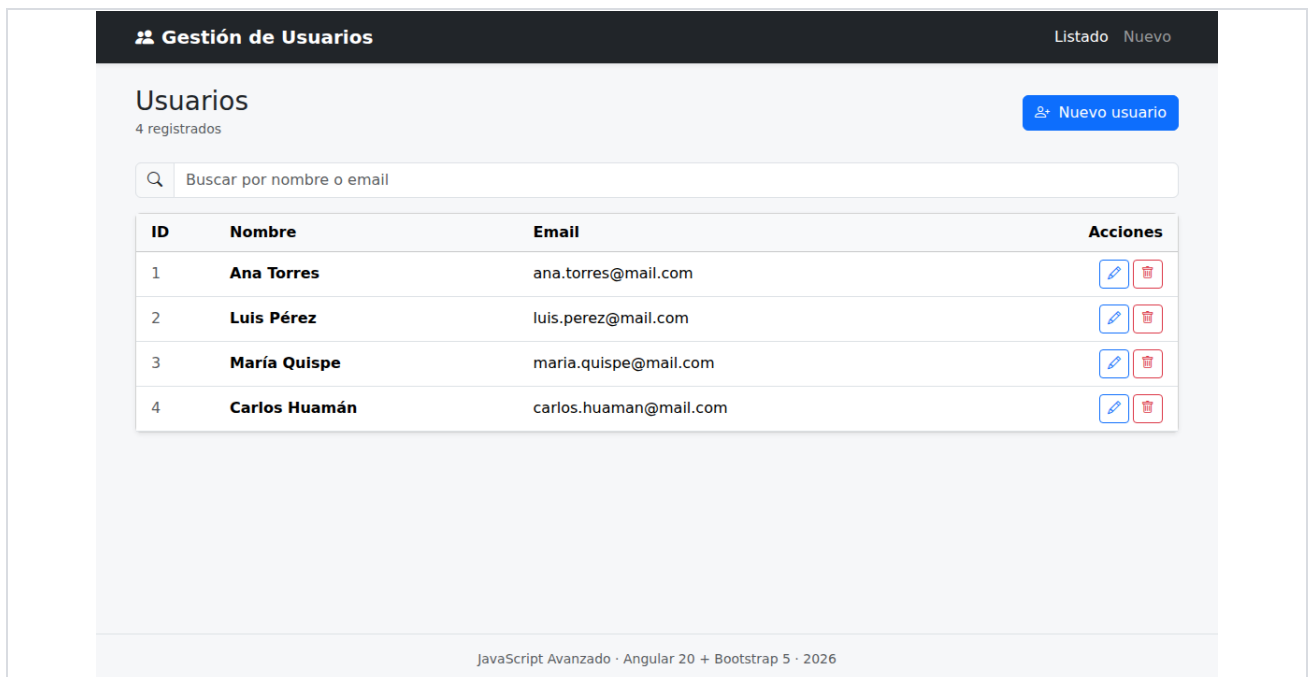
Al finalizar la práctica, **desarrollas una aplicación web en Angular 20** con interfaz en **Bootstrap 5** que consume la API REST de usuarios construida en las guías anteriores. La aplicación lista, busca, crea, edita y elimina usuarios; **valida** los datos en el formulario; **interpreta** las respuestas de error de la API (400, 404, 409) y **comunica** cada resultado al usuario con mensajes claros.

### Lo que vas a construir

Una aplicación de una sola página (SPA) con dos pantallas: el listado de usuarios y el formulario de creación y edición. Todos los datos vienen de la API de Node.js que ya tienes funcionando.

**Figura 1**

*Pantalla de listado de usuarios con búsqueda y acciones*



*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

| Ruta                 | Pantalla             | Operación de la API                     |
|----------------------|----------------------|-----------------------------------------|
| /usuarios            | Listado con búsqueda | GET /api/users · DELETE /api/users/:id  |
| /usuarios/nuevo      | Formulario vacío     | POST /api/users                         |
| /usuarios/:id/editar | Formulario con datos | GET /api/users/:id · PUT /api/users/:id |

## 2. Requisitos previos

| Requisito          | Detalle                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------|
| Node.js            | <b>20.19 o superior</b> (o 22.12+). Angular 20 no funciona con versiones anteriores de Node 20.        |
| API de usuarios    | Funcionando en <code>http://localhost:3000</code> (guía "API REST con Node.js 20, Express 5 y MySQL"). |
| Visual Studio Code | Con la extensión <b>Angular Language Service</b> (autocompletado en las plantillas).                   |
| Conocimientos      | TypeScript básico (tipos, interfaces, clases), HTML y CSS.                                             |

⚠ **Importante:** Comprueba tu versión con `node -v`. Si ves algo como `v20.12.0`, actualiza con `nvm install 20` y `nvm use 20`: `nvm` instalará la última versión 20.x, que es compatible con Angular 20 y con la API.

## 3. Fundamentos

Angular 20 consolida una forma moderna de escribir aplicaciones. Estos son los conceptos que usarás en la práctica y dónde aparecen:

| Concepto                | Qué es                                                                                                                                     | Dónde lo usas                       |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Componentes standalone  | Componentes que declaran sus propias dependencias en <code>imports</code> , sin <code>NgModules</code>                                     | Todos los componentes               |
| Signals                 | Valores reactivos: <code>signal()</code> guarda estado y <code>computed()</code> deriva valores que se recalculan solos (Angular, s. f.-d) | Estado del listado y del formulario |
| Control flow            | Bloques <code>@if</code> , <code>@else</code> y <code>@for</code> directamente en la plantilla (Angular, s. f.-a)                          | Plantillas HTML                     |
| <code>inject()</code>   | Obtiene un servicio sin necesidad de un constructor                                                                                        | Componentes y servicios             |
| <code>HttpClient</code> | Cliente HTTP de Angular; devuelve <code>Observables</code> (Angular, s. f.-c)                                                              | <code>UserService</code>            |
| Formularios reactivos   | El formulario se define en TypeScript con sus validadores (Angular, s. f.-b)                                                               | <code>UserForm</code>               |
| Proxy de desarrollo     | <code>ng serve</code> reenvía las peticiones de <code>/api</code> al backend (Angular, s. f.-e)                                            | <code>proxy.conf.json</code>        |

### ¿Por qué un proxy?

El frontend corre en `http://localhost:4200` y la API en `http://localhost:3000`. Para el navegador son **orígenes distintos**, y la política CORS puede bloquear las peticiones. Con el proxy, Angular recibe la petición a `/api/users` en el mismo puerto 4200 y la reenvía a la API. Para el navegador todo ocurre en el mismo origen, así que no hay bloqueo.

Navegador —GET /api/users—▶ ng serve :4200 —proxy—▶ API Express :3000 —▶ MySQL  
◀— JSON —◀ ◀— JSON —◀

## 4. Desarrollo de la práctica

Sigue los pasos en orden. Mantén la API encendida en otra terminal (`npm run dev` dentro de la carpeta de la API) durante toda la práctica.

### Paso 1. Crear el proyecto

Instala Angular CLI 20 de forma global y crea el proyecto. Ubícate **fuera** de la carpeta de la API:

```
npm install -g @angular/cli@20
ng version
ng new frontend-usuarios --style=css --routing --ssr=false
cd frontend-usuarios
ng serve
```

Si el CLI pregunta por herramientas de IA o por análisis de uso, responde **None** y **No**. Abre <http://localhost:4200>: verás la página de bienvenida de Angular. Detén el servidor con **Ctrl+C**.

✓ **Punto de control:** `ng version` muestra **Angular CLI: 20.x** y la página de bienvenida carga en el navegador.

### Paso 2. Instalar Bootstrap y sus iconos

```
npm install bootstrap bootstrap-icons
```

Registra los estilos en `angular.json`. Busca la propiedad `styles` dentro de `build > options` (y repite lo mismo en `test > options`):

```
angular.json (fragmento)
"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "node_modules/bootstrap-icons/font/bootstrap-icons.css",
  "src/styles.css"
]
```

Solo se usa el CSS de Bootstrap. Las partes interactivas (el modal y los mensajes emergentes) se controlarán con Angular, así no hace falta el JavaScript de Bootstrap. Reemplaza `src/styles.css` y, en `src/index.html`, cambia `lang="en"` por `lang="es"` y el título por **Gestión de Usuarios**:

```
src/styles.css
/* Estilos globales. Bootstrap e iconos se cargan desde angular.json */
body {
  background-color: #f6f7f9;
}
```

💡 **Nota:** Cada vez que modifiques `angular.json` debes **reiniciar** `ng serve` para que tome los cambios.

### Paso 3. Configurar los entornos y el proxy

```
ng generate environments
```

El comando crea dos archivos. En desarrollo la URL es relativa (`/api`) porque pasará por el proxy; en producción apunta a la API real:

**src/environments/environment.development.ts**

```
// Configuración para desarrollo (ng serve)
// '/api' pasa por el proxy (proxy.conf.json) hacia http://localhost:3000
export const environment = {
  production: false,
  apiUrl: '/api',
};
```

**src/environments/environment.ts**

```
// Configuración para producción (ng build)
// Cambia apiUrl por la URL pública de tu API cuando la despliegues.
export const environment = {
  production: true,
  apiUrl: 'http://localhost:3000/api',
};
```

Crea el archivo del proxy en la **raíz** del proyecto (al lado de **package.json**):

**proxy.conf.json**

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false
  }
}
```

Y actívalo en **angular.json**, dentro de **serve**:

**angular.json (fragmento)**

```
"serve": {
  "builder": "@angular/build:dev-server",
  "options": {
    "proxyConfig": "proxy.conf.json"
  },
  ...
}
```

## Paso 4. Generar la estructura del proyecto

Usa el CLI para crear los archivos. En Angular 20 los archivos ya no llevan el sufijo **.component** ni **.service**: el componente **user-list** se llama **user-list.ts**.

```
ng generate interface models/user
ng generate service services/user
ng generate service services/notification
ng generate component pages/user-list
ng generate component pages/user-form
ng generate component shared/notifications
ng generate component shared/confirm-dialog
```

Crea también a mano el archivo **src/app/services/api-error.ts**. Al terminar tendrás esta estructura:

```
src/app/
├── models/user.ts
├── pages/
│   ├── user-form/    (user-form.ts, user-form.html)
│   └── user-list/    (user-list.ts, user-list.html, user-list.css)
├── services/
│   ├── api-error.ts
│   ├── notification.ts
│   └── user.ts
├── shared/
│   ├── confirm-dialog/ (confirm-dialog.ts, confirm-dialog.html)
│   └── notifications/  (notifications.ts, notifications.html)
├── app.config.ts
├── app.routes.ts
└── app.ts / app.html / app.css
```

## Paso 5. Definir el modelo

La interfaz describe exactamente lo que devuelve la API. **UserInput** es el mismo tipo sin el **id**, porque ese valor lo asigna la base de datos:

```
src/app/models/user.ts
// Estructura de un usuario tal como la devuelve la API
export interface User {
  id: number;
  name: string;
  email: string;
}

// Datos que se envían al crear o editar (el id lo asigna la base de datos)
export type UserInput = Omit<User, 'id'>;
```

## Paso 6. Crear el servicio HTTP

El servicio concentra **todas** las llamadas a la API. Los componentes nunca usan **HttpClient** directamente: si mañana cambia la URL, solo se modifica este archivo.

```
src/app/services/user.ts
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { User, UserInput } from '../../models/user';

@Injectable({ providedIn: 'root' })
export class UserService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/users`;

  getAll(): Observable<User[]> {
    return this.http.get<User[]>(this.url);
  }

  getById(id: number): Observable<User> {
    return this.http.get<User>(`${this.url}/${id}`);
  }

  create(data: UserInput): Observable<User> {
    return this.http.post<User>(this.url, data);
  }

  update(id: number, data: Partial<UserInput>): Observable<User> {
    return this.http.put<User>(`${this.url}/${id}`, data);
  }
}
```

```
delete(id: number): Observable<void> {
  return this.http.delete<void>(`${this.url}/${id}`);
}
}
```

Para que `HttpClient` esté disponible hay que registrarlo en `app.config.ts`. Aprovecha para activar `withComponentInputBinding()`, que entrega el parámetro `:id` de la ruta directamente como un `input` del componente:

```
src/app/app.config.ts
import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection } from '@angular/core';
import { provideHttpClient, withFetch } from '@angular/common/http';
import { provideRouter, withComponentInputBinding } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withFetch()),
  ],
};
```

**Nota:** Los métodos del servicio devuelven un **Observable**: la petición HTTP no se envía hasta que alguien llama a `.subscribe()`.

## Paso 7. Interpretar los errores y mostrar notificaciones

La API responde los errores con `{ "error": "..." }` o con `{ "errors": [...] }`. Esta función los convierte en un texto que se puede mostrar al usuario, y reconoce cuándo la API ni siquiera responde:

```
src/app/services/api-error.ts
import { HttpResponseError } from '@angular/common/http';

const SIN_CONEXION = 'No se pudo conectar con la API. Verifica que esté corriendo en el puerto 3000.';

// Convierte cualquier error HTTP en un mensaje legible para el usuario.
// La API responde { error: '...' } o { errors: ['...', '...'] }.
export function apiErrorMessage(err: unknown): string {
  if (!(err instanceof HttpResponseError)) return 'Ocurrió un error inesperado.';

  // status 0: el navegador no llegó al servidor.
  // 502-504: el proxy de desarrollo (ng serve) no pudo contactar a la API.
  if (err.status === 0 || (err.status >= 502 && err.status <= 504)) return SIN_CONEXION;

  const body = typeof err.error === 'object' ? err.error : null;
  if (body?.errors?.length) return body.errors.join('. ');
  if (body?.error) return body.error;

  // Un 500 sin cuerpo JSON viene del proxy, no de nuestra API
  if (err.status >= 500) return SIN_CONEXION;
  return `Error ${err.status}: ${err.statusText}`;
}
```

El servicio de notificaciones guarda la lista de mensajes en un **signal**. Cualquier componente puede llamar a `success()` o `error()`, y el mensaje desaparece solo a los 4 segundos:

**src/app/services/notification.ts**

```
import { Injectable, signal } from '@angular/core';

export interface Notification {
  id: number;
  type: 'success' | 'danger';
  text: string;
}

// Mensajes emergentes (toasts) que se cierran solos a los 4 segundos
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private nextId = 1;
  readonly items = signal<Notification[]>([]);

  success(text: string) {
    this.show('success', text);
  }

  error(text: string) {
    this.show('danger', text);
  }

  dismiss(id: number) {
    this.items.update((list) => list.filter((n) => n.id !== id));
  }

  private show(type: Notification['type'], text: string) {
    const id = this.nextId++;
    this.items.update((list) => [...list, { id, type, text }]);
    setTimeout(() => this.dismiss(id), 4000);
  }
}
```

El componente que los dibuja usa los *\*toasts\** de Bootstrap:

**src/app/shared/notifications/notifications.ts**

```
import { Component, inject } from '@angular/core';
import { NotificationService } from '../../services/notification';

@Component({
  selector: 'app-notifications',
  templateUrl: './notifications.html',
})
export class Notifications {
  protected readonly notifications = inject(NotificationService);
}
```

**src/app/shared/notifications/notifications.html**

```
<div class="toast-container position-fixed top-0 end-0 p-3">
  @for (n of notifications.items(); track n.id) {
    <div class="toast show align-items-center border-0 text-bg-{{ n.type }}" role="alert">
      <div class="d-flex">
        <div class="toast-body">
          <i class="bi me-2" [class.bi-check-circle-fill]="n.type === 'success'"
            [class.bi-exclamation-triangle-fill]="n.type === 'danger'"></i>
          {{ n.text }}
        </div>
        <button type="button" class="btn-close btn-close-white me-2 m-auto"
          aria-label="Cerrar" (click)="notifications.dismiss(n.id)"></button>
      </div>
    </div>
  }
</div>
```

## Paso 8. Crear el modal de confirmación

Un componente reutilizable que recibe datos con `input()` y avisa la decisión con `output()`. Se muestra solo cuando `open` es verdadero:

**src/app/shared/confirm-dialog/confirm-dialog.ts**

```
import { Component, input, output } from '@angular/core';

// Ventana modal de confirmación controlada por Angular (sin el JavaScript de Bootstrap)
@Component({
  selector: 'app-confirm-dialog',
  templateUrl: './confirm-dialog.html',
})
export class ConfirmDialog {
  readonly open = input(false);
  readonly title = input('Confirmar');
  readonly message = input('');
  readonly confirmText = input('Aceptar');
  readonly busy = input(false);

  readonly confirmed = output<void>();
  readonly cancelled = output<void>();
}
```

**src/app/shared/confirm-dialog/confirm-dialog.html**

```
@if (open()) {
  <div class="modal d-block" tabindex="-1" role="dialog" aria-modal="true">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title">{{ title() }}</h5>
          <button type="button" class="btn-close" aria-label="Cerrar"
(click)="cancelled.emit()"></button>
        </div>
        <div class="modal-body">
          <p class="mb-0">{{ message() }}</p>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-outline-secondary" (click)="cancelled.emit()"
[disabled]="busy()">
            Cancelar
          </button>
          <button type="button" class="btn btn-danger" (click)="confirmed.emit()" [disabled]="busy()">
            @if (busy()) {
              <span class="spinner-border spinner-border-sm me-1"></span>
            }
            {{ confirmText() }}
          </button>
        </div>
      </div>
    </div>
  </div>
</div>
<div class="modal-backdrop show"></div>
}
```

## Paso 9. Construir el listado

Observa cómo se modela el estado con signals: `users`, `loading`, `error` y `search` son valores que cambian, y `filtered` es un `computed()` que se recalcula solo cada vez que cambia la lista o el texto de búsqueda. No hace falta volver a llamar a la API para filtrar.

**src/app/pages/user-list/user-list.ts**

```
import { Component, OnInit, computed, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { User } from '../../models/user';
```

```
import { UserService } from '../services/user';
import { NotificationService } from '../services/notification';
import { apiErrorMessage } from '../services/api-error';
import { ConfirmDialog } from '../shared/confirm-dialog/confirm-dialog';

@Component({
  selector: 'app-user-list',
  imports: [RouterLink, ConfirmDialog],
  templateUrl: './user-list.html',
  styleUrls: ['./user-list.css'],
})
export class UserList implements OnInit {
  private readonly userService = inject(UserService);
  private readonly notify = inject(NotificationService);

  // Estado de la pantalla con signals
  readonly users = signal<User[]>([]);
  readonly loading = signal(true);
  readonly error = signal<string | null>(null);
  readonly search = signal('');
  readonly toDelete = signal<User | null>(null);
  readonly deleting = signal(false);

  // Lista filtrada: se recalcula sola cuando cambian users o search
  readonly filtered = computed(() => {
    const term = this.search().trim().toLowerCase();
    if (!term) return this.users();
    return this.users().filter(
      (u) => u.name.toLowerCase().includes(term) || u.email.toLowerCase().includes(term),
    );
  });

  ngOnInit(): void {
    this.load();
  }

  load(): void {
    this.loading.set(true);
    this.error.set(null);
    this.userService.getAll().subscribe({
      next: (users) => {
        this.users.set(users);
        this.loading.set(false);
      },
      error: (err) => {
        this.error.set(apiErrorMessage(err));
        this.loading.set(false);
      },
    });
  }

  onSearch(event: Event): void {
    this.search.set((event.target as HTMLInputElement).value);
  }

  askDelete(user: User): void {
    this.toDelete.set(user);
  }

  confirmDelete(): void {
    const user = this.toDelete();
    if (!user) return;
    this.deleting.set(true);
    this.userService.delete(user.id).subscribe({
      next: () => {
        this.users.update((list) => list.filter((u) => u.id !== user.id));
        this.notify.success(`Usuario "${user.name}" eliminado.`);
      },
    });
  }
}
```

```

        this.closeDialog();
      },
      error: (err) => {
        this.notify.error(apiErrorMessage(err));
        this.closeDialog();
      },
    });
  }

  closeDialog(): void {
    this.toDelete.set(null);
    this.deleting.set(false);
  }
}

```

La plantilla usa `@if` / `@else if` / `@else` para los cuatro estados posibles de la pantalla (cargando, error, vacío y con datos) y `@for` con `track` para dibujar las filas:

```

src/app/pages/user-list/user-list.html
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4">
  <div>
    <h1 class="h3 mb-0">Usuarios</h1>
    <small class="text-body-secondary">{{ users().length }} registrados</small>
  </div>
  <a routerLink="/usuarios/nuevo" class="btn btn-primary">
    <i class="bi bi-person-plus me-1"></i> Nuevo usuario
  </a>
</div>

<div class="input-group mb-3">
  <span class="input-group-text"><i class="bi bi-search"></i></span>
  <input type="search" class="form-control" placeholder="Buscar por nombre o email"
    [value]="search()" (input)="onSearch($event)" aria-label="Buscar" />
</div>

@if (loading()) {
  <div class="text-center py-5">
    <div class="spinner-border text-primary" role="status"></div>
    <p class="mt-2 text-body-secondary">Cargando usuarios...</p>
  </div>
} @else if (error()) {
  <div class="alert alert-danger d-flex justify-content-between align-items-center" role="alert">
    <span><i class="bi bi-exclamation-triangle-fill me-2"></i>{{ error() }}</span>
    <button class="btn btn-sm btn-outline-danger" (click)="load()">Reintentar</button>
  </div>
} @else if (filtered().length === 0) {
  <div class="text-center py-5 text-body-secondary">
    <i class="bi bi-people display-4"></i>
    <p class="mt-2">
      {{ search() ? 'Ningún usuario coincide con la búsqueda.' : 'Aún no hay usuarios. Crea el
primer.' }}
    </p>
  </div>
} @else {
  <div class="card shadow-sm">
    <div class="table-responsive">
      <table class="table table-hover align-middle mb-0">
        <thead class="table-light">
          <tr>
            <th scope="col" class="ps-3">ID</th>
            <th scope="col">Nombre</th>
            <th scope="col">Email</th>
            <th scope="col" class="text-end pe-3">Acciones</th>
          </tr>
        </thead>
        <tbody>
          @for (user of filtered(); track user.id) {

```

### Figura 2

 **Gestión de Usuarios**

Listado Nuevo

## Usuarios

4 registrados



quispe



| ID | Nombre              | Email                 | Acciones                                                                                                                                                                                                    |
|----|---------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3  | <b>María Quispe</b> | maria.quispe@mail.com | <div><div></div><div></div></div> |

Guía práctica · Frontend en Angular 20 con Bootstrap 5

## Paso 10. Construir el formulario de creación y edición

Un solo componente sirve para crear y para editar: si la ruta trae un **id**, carga el usuario y al guardar hace **PUT**; si no, hace **POST**. Los validadores replican las reglas de la API (nombre de 2 a 100 caracteres y email con el mismo patrón), así el usuario ve el error **antes** de enviar.

**src/app/pages/user-form/user-form.ts**

```
import { Component, OnInit, computed, inject, input, signal } from '@angular/core';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { HttpResponse } from '@angular/common/http';
import { Router, RouterLink } from '@angular/router';
import { UserService } from '../../services/user';
import { NotificationService } from '../../services/notification';
import { apiErrorMessage } from '../../services/api-error';

// Mismo formato de email que valida la API
const EMAIL_PATTERN = /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/;

@Component({
  selector: 'app-user-form',
  imports: [ReactiveFormsModule, RouterLink],
  templateUrl: './user-form.html',
})
export class UserForm implements OnInit {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly userService = inject(UserService);
  private readonly notify = inject(NotificationService);
  private readonly router = inject(Router);

  // Parámetro :id de la ruta (llega gracias a withComponentInputBinding)
  readonly id = input<string>();
  readonly isEdit = computed(() => !!this.id());

  readonly loading = signal(false);
  readonly saving = signal(false);
  readonly serverError = signal<string | null>(null);

  readonly form = this.fb.group({
    name: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(100)]],
    email: ['', [Validators.required, Validators.pattern(EMAIL_PATTERN), Validators.maxLength(150)]],
  });

  ngOnInit(): void {
    if (!this.isEdit()) return;
    this.loading.set(true);
    this.userService.getById(Number(this.id())).subscribe({
      next: (user) => {
        this.form.setValue({ name: user.name, email: user.email });
        this.loading.set(false);
      },
      error: (err) => {
        this.notify.error(apiErrorMessage(err));
        this.router.navigate(['/usuarios']);
      },
    });
  }

  // true si el campo tiene errores y el usuario ya lo tocó
  invalid(field: 'name' | 'email'): boolean {
    const control = this.form.controls[field];
    return control.invalid && (control.touched || control.dirty);
  }

  save(): void {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
    }
  }
}
```

```

    return;
  }
  this.saving.set(true);
  this.serverError.set(null);
  const data = this.form.getRawValue();
  const request = this.isEdit()
    ? this.userService.update(Number(this.id()), data)
    : this.userService.create(data);

  request.subscribe({
    next: (user) => {
      this.notify.success(this.isEdit() ? `Usuario "${user.name}" actualizado.` : `Usuario
"${user.name}" creado.`);
      this.router.navigate(['/usuarios']);
    },
    error: (err: HttpResponse) => {
      this.saving.set(false);
      if (err.status === 409) {
        // Email duplicado: se marca el error en el propio campo
        this.form.controls.email.setErrors({ duplicate: true });
      } else {
        this.serverError.set(apiErrorMessage(err));
      }
    },
  });
}
}
}

```

Fíjate en el manejo del **409**: en lugar de mostrar una alerta genérica, el error se coloca en el propio campo de email con `setErrors({ duplicate: true })`, y la plantilla lo muestra debajo del campo.

#### src/app/pages/user-form/user-form.html

```

<nav aria-label="breadcrumb">
  <ol class="breadcrumb">
    <li class="breadcrumb-item"><a routerLink="/usuarios">Usuarios</a></li>
    <li class="breadcrumb-item active" aria-current="page">{{ isEdit() ? 'Editar' : 'Nuevo' }}</li>
  </ol>
</nav>

<div class="row justify-content-center">
  <div class="col-lg-6 col-md-8">
    <div class="card shadow-sm">
      <div class="card-header bg-white">
        <h1 class="h5 mb-0">
          <i class="bi me-1" [class.bi-pencil-square]="isEdit()" [class.bi-person-plus]="!isEdit()"></i>
          {{ isEdit() ? 'Editar usuario #' + id() : 'Nuevo usuario' }}
        </h1>
      </div>
      <div class="card-body">
        @if (loading()) {
          <div class="text-center py-4"><div class="spinner-border text-primary"></div></div>
        } @else {
          @if (serverError()) {
            <div class="alert alert-danger" role="alert">
              <i class="bi bi-exclamation-triangle-fill me-2"></i>{{ serverError() }}
            </div>
          }

          <form [formGroup]="form" (ngSubmit)="save()" novalidate>
            <div class="mb-3">
              <label for="name" class="form-label">Nombre</label>
              <input id="name" type="text" class="form-control" formControlName="name"
                [class.is-invalid]="invalid('name')" placeholder="Ej.: Ana Torres" />
              <div class="invalid-feedback">
                @if (form.controls.name.hasError('required')) { El nombre es obligatorio. }
              </div>
            </div>
          </form>
        }
      </div>
    </div>
  </div>
</div>

```

```

        @else if (form.controls.name.hasError('minlength')) { Debe tener al menos 2 caracteres.
    }
    @else if (form.controls.name.hasError('maxlength')) { Máximo 100 caracteres. }
</div>
</div>

<div class="mb-4">
    <label for="email" class="form-label">Email</label>
    <input id="email" type="email" class="form-control" formControlName="email"
        [class.is-invalid]="invalid('email')" placeholder="Ej.: ana@mail.com" />
    <div class="invalid-feedback">
        @if (form.controls.email.hasError('required')) { El email es obligatorio. }
        @else if (form.controls.email.hasError('pattern')) { Ingresa un email válido. }
        @else if (form.controls.email.hasError('maxlength')) { Máximo 150 caracteres. }
        @else if (form.controls.email.hasError('duplicate')) { Ya existe un usuario con ese
email. }
    </div>
</div>

<div class="d-flex gap-2 justify-content-end">
    <a routerLink="/usuarios" class="btn btn-outline-secondary">Cancelar</a>
    <button type="submit" class="btn btn-primary" [disabled]="saving()">
        @if (saving()) {
            <span class="spinner-border spinner-border-sm me-1"></span>
        } @else {
            <i class="bi bi-save me-1"></i>
        }
        Guardar
    </button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>

```

**Figura 3**

*Validación del formulario al intentar guardar sin datos*

The screenshot shows a web application titled "Gestión de Usuarios" with a dark header. In the top right of the header are links for "Listado" and "Nuevo". Below the header, there's a breadcrumb "Usuarios / Nuevo". The main content area features a modal form titled "Nuevo usuario". This form contains two input fields: "Nombre" and "Email". The "Nombre" field has a red border and a red error message "El nombre es obligatorio." below it. The "Email" field also has a red border and a red error message "El email es obligatorio." below it. At the bottom of the form are two buttons: "Cancelar" (light gray) and "Guardar" (blue with a save icon).

*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

**Figura 4**

Error 409 de la API mostrado en el campo de email

*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

**Seguridad:** La validación del frontend **mejora la experiencia**, pero **no protege** la API: cualquiera puede enviar peticiones con REST Client o Postman saltándose Angular. Por eso la validación del backend de la guía anterior sigue siendo obligatoria.

## Paso 11. Configurar rutas y diseño general

**src/app/app.routes.ts**

```
import { Routes } from '@angular/router';
import { UserList } from '../pages/user-list/user-list';
import { UserForm } from '../pages/user-form/user-form';

export const routes: Routes = [
  { path: '', redirectTo: 'usuarios', pathMatch: 'full' },
  { path: 'usuarios', component: UserList, title: 'Usuarios' },
  { path: 'usuarios/nuevo', component: UserForm, title: 'Nuevo usuario' },
  { path: 'usuarios/:id/editar', component: UserForm, title: 'Editar usuario' },
  { path: '**', redirectTo: 'usuarios' },
];
```

El componente raíz dibuja la barra de navegación, el contenido de cada ruta (`<router-outlet />`), el pie de página y el contenedor de notificaciones:

#### src/app/app.ts

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { Notifications } from '../shared/notifications/notifications';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive, Notifications],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly year = new Date().getFullYear();
}
```

#### src/app/app.html

```
<nav class="navbar navbar-expand navbar-dark bg-dark shadow-sm">
  <div class="container">
    <a class="navbar-brand fw-semibold" routerLink="/">
      <i class="bi bi-people-fill me-2"></i>Gestión de Usuarios
    </a>
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link" routerLink="/usuarios" routerLinkActive="active"
          [routerLinkActiveOptions]="{ exact: true }">Listado</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/usuarios/nuevo" routerLinkActive="active">Nuevo</a>
      </li>
    </ul>
  </div>
</nav>

<main class="container py-4 flex-grow-1">
  <router-outlet />
</main>

<footer class="border-top py-3 text-center text-body-secondary small">
  JavaScript Avanzado · Angular 20 + Bootstrap 5 · {{ year }}
</footer>

<app-notifications />
```

#### src/app/app.css

```
:host {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
```

## Paso 12. Ejecutar y probar la aplicación

Con la API encendida, ejecuta en la carpeta del frontend:

```
npm start
```

Abre <http://localhost:4200> y realiza estas pruebas en orden. Anota en **Obtenido** lo que observas:

| # | Acción                                                          | Resultado esperado                      | Obtenido |
|---|-----------------------------------------------------------------|-----------------------------------------|----------|
| 1 | Abrir <a href="http://localhost:4200">http://localhost:4200</a> | Redirige a /usuarios y muestra la tabla |          |
| 2 | Escribir parte de un nombre en el buscador                      | La tabla se filtra al instante          |          |

| #  | Acción                                | Resultado esperado                                         | Obtenido |
|----|---------------------------------------|------------------------------------------------------------|----------|
| 3  | Nuevo usuario › Guardar sin datos     | Mensajes "obligatorio" en ambos campos                     |          |
| 4  | Nombre "A" y email "correo-malo"      | Mensajes de longitud y de email inválido                   |          |
| 5  | Usar el email de un usuario existente | "Ya existe un usuario con ese email" (409)                 |          |
| 6  | Crear un usuario con datos válidos    | Vuelve al listado con notificación verde                   |          |
| 7  | Editar un usuario y guardar           | Datos actualizados y notificación verde                    |          |
| 8  | Eliminar › Cancelar                   | El modal se cierra y el usuario sigue                      |          |
| 9  | Eliminar › Eliminar                   | El usuario desaparece y aparece notificación               |          |
| 10 | Abrir /usuarios/999/editar            | Vuelve al listado con "Usuario no encontrado"              |          |
| 11 | Detener la API y recargar             | Alerta "No se pudo conectar con la API" y botón Reintentar |          |
| 12 | Encender la API y pulsar Reintentar   | La tabla vuelve a cargar                                   |          |

**Figura 5**

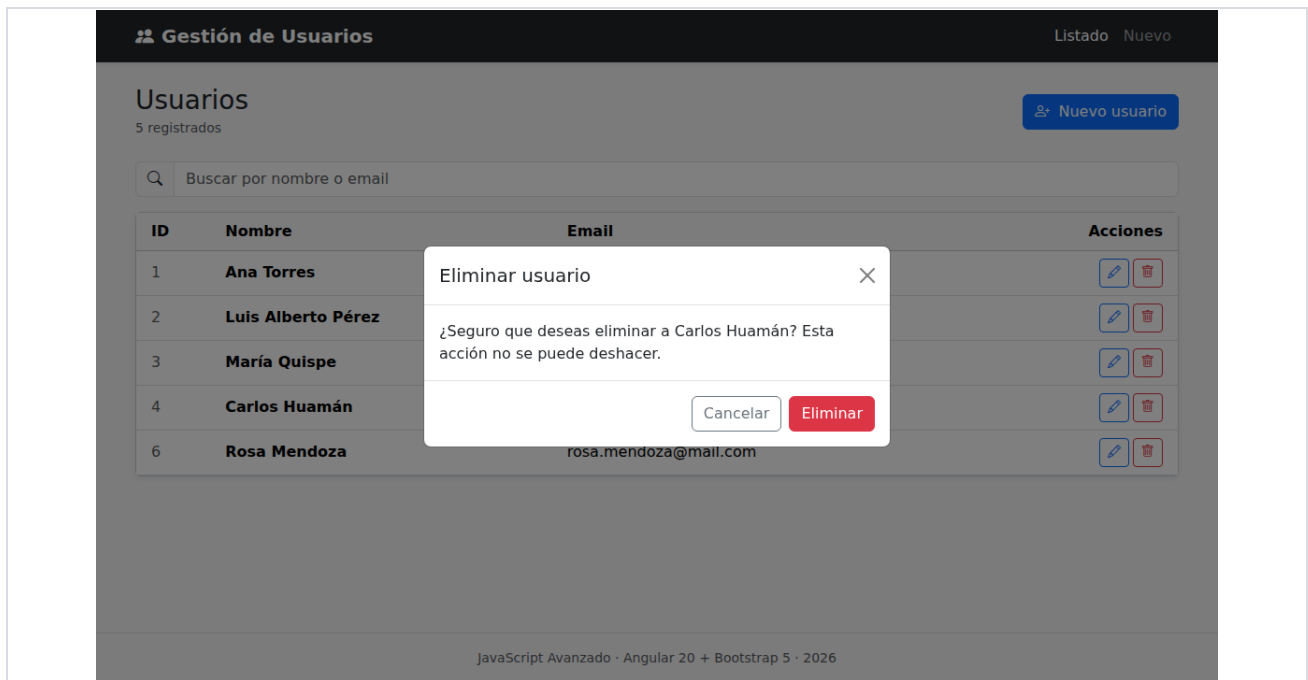
*Notificación de éxito tras crear un usuario*

The screenshot shows a web application interface for user management. At the top, there's a dark header with the title "Gestión de Usuarios". A green notification banner at the top right states "Usuario 'Rosa Mendoza' creado." with a close button. Below the header, the main content area is titled "Usuarios" with a subtitle "5 registrados". A blue button labeled "Nuevo usuario" is in the top right. A search bar with the placeholder "Buscar por nombre o email" is present. Below it is a table with columns: ID, Nombre, Email, and Acciones. The table lists five users: Ana Torres, Luis Pérez, María Quispe, Carlos Huamán, and Rosa Mendoza. Each user has edit and delete icons in the Acciones column. The footer of the application indicates "JavaScript Avanzado · Angular 20 + Bootstrap 5 · 2026".

*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

**Figura 6**

Modal de confirmación antes de eliminar



*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

**Figura 7**

Mensaje mostrado cuando la API no está disponible



*Nota.* Captura de la aplicación desarrollada en esta guía. Elaboración propia.

**Nota:** Abre las herramientas de desarrollo del navegador (**F12**) en la pestaña **Red** mientras haces las pruebas. Verás cada petición a **/api/users**, su método y su código de estado: es la mejor forma de entender qué ocurre entre el frontend y la API.

✓ **Punto de control:** Las 12 pruebas se comportan como se espera. Guarda capturas de las pruebas 3, 5, 9 y 11.

### Paso 13. Ejecutar las pruebas unitarias

Las pruebas del servicio usan `HttpTestingController`, que intercepta las peticiones: se verifica el método y la URL sin necesidad de la API real. Reemplaza el contenido de `src/app/services/user.spec.ts`:

```
src/app/services/user.spec.ts
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
import { UserService } from './user';
import { environment } from '../../environments/environment';

const URL = `${environment.apiUrl}/users`;

describe('UserService', () => {
  let service: UserService;
  let http: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient(), provideHttpClientTesting()],
    });
    service = TestBed.inject(UserService);
    http = TestBed.inject(HttpTestingController);
  });

  afterEach(() => http.verify());

  it('getAll hace GET a /users', () => {
    service.getAll().subscribe((users) => expect(users.length).toBe(1));
    const req = http.expectOne(URL);
    expect(req.request.method).toBe('GET');
    req.flush([{ id: 1, name: 'Ana', email: 'ana@mail.com' }]);
  });

  it('create hace POST con el cuerpo', () => {
    service.create({ name: 'Luis', email: 'luis@mail.com' }).subscribe();
    const req = http.expectOne(URL);
    expect(req.request.method).toBe('POST');
    expect(req.request.body).toEqual({ name: 'Luis', email: 'luis@mail.com' });
    req.flush({ id: 2, name: 'Luis', email: 'luis@mail.com' });
  });

  it('update hace PUT a /users/:id', () => {
    service.update(5, { name: 'Ana María' }).subscribe();
    const req = http.expectOne(`${URL}/5`);
    expect(req.request.method).toBe('PUT');
    req.flush({ id: 5, name: 'Ana María', email: 'ana@mail.com' });
  });

  it('delete hace DELETE a /users/:id', () => {
    service.delete(5).subscribe();
    const req = http.expectOne(`${URL}/5`);
    expect(req.request.method).toBe('DELETE');
    req.flush(null, { status: 204, statusText: 'No Content' });
  });
});
```

Ejecuta `npm test`. Se abrirá Chrome y la terminal mostrará el resultado. El proyecto de referencia incluye además pruebas para el listado (`user-list.spec.ts`) y para `apiErrorMessage` (`api-error.spec.ts`).

✓ **Punto de control:** `npm test` termina con **TOTAL: ... SUCCESS** y ninguna prueba fallida.

## Paso 14. Compilar para producción

```
ng build
```

El resultado queda en `dist/frontend-usuarios/browser` y se puede publicar en cualquier servidor web. En producción **no hay proxy**: el frontend llamará a la URL de `environment.ts`, así que la API debe permitir el origen del frontend en CORS. En el `.env` de la API, agrega el dominio donde publiques el frontend:

```
.env de la API
CORS_ORIGIN=http://localhost:4200,https://mi-frontend.com
```

## 5. Errores frecuentes y cómo resolverlos

| Mensaje o síntoma                           | Causa probable                                        | Solución                                                 |
|---------------------------------------------|-------------------------------------------------------|----------------------------------------------------------|
| "Node.js version v20.x detected" al usar ng | Node 20 anterior a 20.19                              | nvm install 20 y nvm use 20                              |
| La página se ve sin estilos de Bootstrap    | angular.json sin los estilos o ng serve sin reiniciar | Revisa el Paso 2 y reinicia ng serve                     |
| NG0201: No provider for HttpClient          | Falta provideHttpClient()                             | Revisa app.config.ts (Paso 6)                            |
| Can't bind to 'formGroup'                   | Falta ReactiveFormsModule en imports                  | Agrégallo al componente UserForm                         |
| 404 al pedir /api/users                     | El proxy no está configurado                          | Revisa proxy.conf.json y angular.json; reinicia ng serve |
| Error de CORS en la consola                 | Se llama a localhost:3000 directo en desarrollo       | Usa apiUrl: '/api' en environment.development.ts         |
| "No se pudo conectar con la API"            | La API o MySQL están detenidos                        | Inicia MySQL y ejecuta npm run dev en la API             |
| El formulario de edición aparece vacío      | Falta withComponentInputBinding()                     | Agrégallo a provideRouter en app.config.ts               |

## 6. Lista de cotejo

**Entregable:** el enlace a tu repositorio de GitHub con el proyecto `frontend-usuarios` (sin `node_modules`) y un PDF con la tabla del Paso 12 completa, las capturas solicitadas, la salida de `npm test` y tus respuestas a las preguntas de reflexión. Nombre del PDF: `Apellido_Nombre_Angular.pdf`.

El docente verificará cada criterio. Antes de entregar, márcalo tú mismo en la columna **Autoevaluación**.

| N.º | Criterio                                                                                                      | Puntaje | Autoevaluación | Docente |
|-----|---------------------------------------------------------------------------------------------------------------|---------|----------------|---------|
| 1   | El proyecto usa Angular 20 con componentes standalone y Bootstrap 5 correctamente integrado.                  | 2       |                |         |
| 2   | El proxy de desarrollo funciona: las peticiones van a <code>/api</code> sin errores de CORS.                  | 2       |                |         |
| 3   | Todas las llamadas HTTP están en <code>UserService</code> ; los componentes no usan <code>HttpClient</code> . | 2       |                |         |

| N.º | Criterio                                                                                                              | Puntaje   | Autoevaluación | Docente |
|-----|-----------------------------------------------------------------------------------------------------------------------|-----------|----------------|---------|
| 4   | El listado muestra los estados de carga, error, vacío y con datos, y la búsqueda filtra con <code>computed()</code> . | 3         |                |         |
| 5   | El formulario reactivo crea y edita, con validaciones coherentes con la API.                                          | 3         |                |         |
| 6   | Los errores de la API (400, 404, 409 y sin conexión) se muestran con mensajes claros.                                 | 3         |                |         |
| 7   | La eliminación pide confirmación y actualiza la tabla sin recargar la página.                                         | 2         |                |         |
| 8   | <code>npm test</code> pasa sin fallos.                                                                                | 1         |                |         |
| 9   | Las evidencias y las respuestas a las preguntas de reflexión están completas.                                         | 2         |                |         |
|     | <b>Total</b>                                                                                                          | <b>20</b> |                |         |

## 7. Preguntas de reflexión

Responde brevemente, con tus propias palabras, al final de tu PDF de evidencias:

1. ¿Qué problema resuelve el proxy de desarrollo? ¿Por qué en producción hay que configurar CORS en la API?
2. ¿Qué ventaja tiene `filtered = computed(...)` frente a filtrar la lista cada vez que el usuario escribe con un método que modifique `users`?
3. Si ya validas el email en Angular, ¿por qué la API también debe validarlo? Da un ejemplo de cómo se podría saltar la validación del frontend.
4. ¿Por qué el error 409 se muestra en el campo de email y no en una alerta general? ¿Qué gana el usuario?
5. ¿Por qué conviene que los componentes usen `UserService` en lugar de llamar a `HttpClient` directamente?

## 8. Referencias

Angular. (s. f.-a). *Control flow*. <https://angular.dev/guide/templates/control-flow>

Angular. (s. f.-b). *Reactive forms*. <https://angular.dev/guide/forms/reactive-forms>

Angular. (s. f.-c). *Setting up HttpClient*. <https://angular.dev/guide/http/setup>

Angular. (s. f.-d). *Signals: Overview*. <https://angular.dev/guide/signals>

Angular. (s. f.-e). *Serving Angular apps for development: Proxying to a backend server*.  
<https://angular.dev/tools/cli/serve>

Bootstrap. (s. f.). *Get started with Bootstrap (v5.3)*. <https://getbootstrap.com/docs/5.3/getting-started/introduction/>